



Kai: A Typed Decision Model over Evidence

Hanzo AI Research

Hanzo AI

2026-09-27 Code at [hanzoai/decision 4bdc2e6](#)

Kai: A Typed Decision Model over Evidence

Factorized choice, typed-variable diffusion, and decisions

as versioned programs under deterministic policy

Hanzo AI Research*
Hanzo Industries Inc (Techstars '17)
San Francisco, California
<https://hanzo.ai>

2026-09-27 Code at [hanzoai/decision](https://github.com/hanzoai/decision) 4bdc2e6

Abstract

Agents, controllers and engineering studies ask many small questions of explicit state. Kai answers them as typed decisions, a **choice**, **score** or **noul** over options the caller lists, returned as a calibrated distribution with no generated text. Kai is one model on one checkpoint line: a multilingual encoder, adapters that bring sensor streams with their metadata, audio, images and video into it, and a typed-variable decoder. Options are encoded alone and scored against the evidence by two cross-attention blocks, so option count, order and question count are not bounded by one sequence; **noul** is label-symmetric by construction, **score** a cumulative-link model, and a large choice is narrowed by retrieval that keeps the mass it cuts. A program's dependent questions are typed variables, decoded as masked slots, refined in waves bounded by dependency depth and projected exactly onto the program's rules. Programs are versioned DAGs whose verdicts join a deterministic policy on `ALLOW < ASK < DENY`, so no model loosens policy, and each run is a hashed package that replays, diffs, refreshes and says what flips the decision. Kai trains in pure Rust, bf16 over F32 masters, as local SGD across Metal, CUDA and ROCm machines, on data classed by license and held behind a contamination barrier. We report what went wrong: our first stage-A run lost the encoder's ability to relate two texts, and we give the measurements, the cause and the fix. Results against Laya and TypeSafe's Jev on a frozen 62-suite harness, on capabilities they lack by construction, and on speed are written into the paper by the benchmark that measures them.

*Corresponding author: research@hanzo.ai

Contents

1	Introduction	4
2	Typed decisions	5
2.1	What an answer is sure of	6
3	Kai	6
3.1	Evidence	6
3.2	Factorized choice	8
3.3	Typed-variable diffusion	9
3.4	One checkpoint line	10
4	Training	10
4.1	Stages	10
4.2	Local SGD across devices	11
4.3	Data, licenses and the barrier	12
5	What stage A taught	13
5.1	The encoder stopped relating two texts	13
5.2	Laya had seen half the typed-decisions test set, reworded	15
5.3	The score head	15
5.4	Other defects, each fixed	16
6	Policy is the authority	16
7	Programs and packages	17
7.1	The Decision Package	18
7.2	What flips the decision	19
7.3	Studies, SysML and the engineering graph	19
7.4	Kai controls Enso	20
7.5	The wire and where it runs	20
8	Evaluation	21
8.1	Accuracy on the frozen harness	21
8.2	Capabilities the baselines lack by construction	22
8.3	Speed	23
9	Limits	26
10	Related work	27
11	Conclusion	28
A	The wire	31
B	A Decision Program	32
C	MASSIVE by language	32

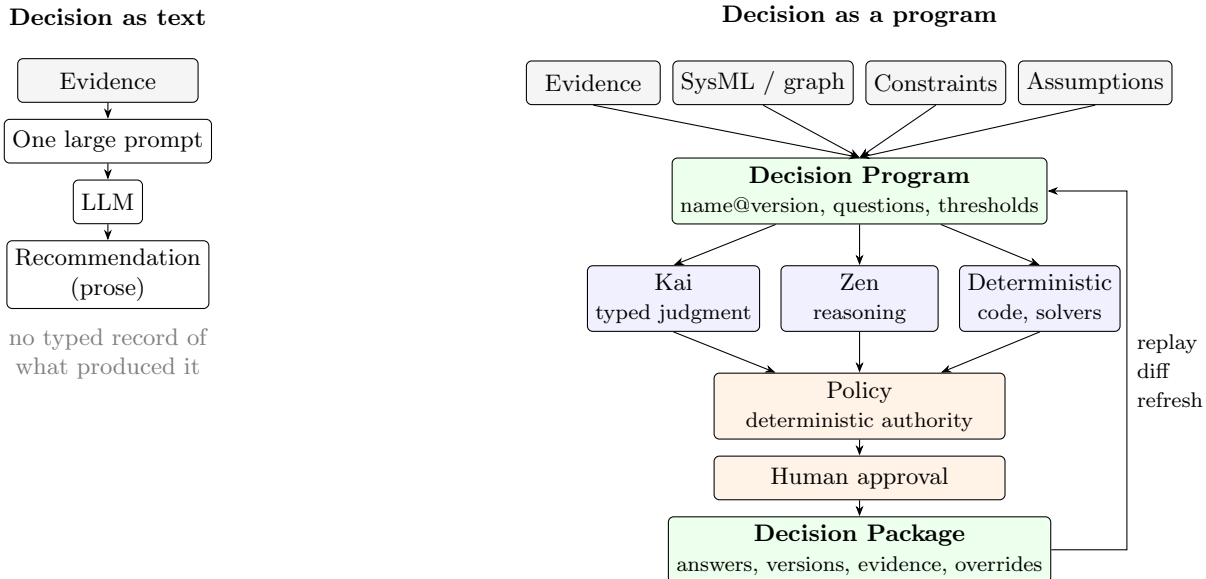


Figure 1: Left: the usual shape, in which one generative call turns evidence into a recommendation and nothing typed records how. Right: Hanzo Decision. A versioned program DAG states the questions; Kai answers the bounded judgments, Zen the open reasoning, deterministic code and solvers the arithmetic; policy and a person hold authority; every run produces a hashed package that can be replayed, diffed and refreshed.

1 Introduction

Agents ask the same small questions thousands of times per session: which context the next turn sees, which tool fits, whether a command may run [20]. A generative model answers in prose with no probability to threshold; a fixed rule sees only what its author foresaw. A *typed decision* sits between: the caller states a question, lists the admissible answers and passes the evidence, and the model returns a distribution over exactly those answers, which can be type-checked, thresholded, logged and replayed.

Three planes: the *generative* writes (Zen), the *decision* answers typed questions over evidence (Kai), and the *policy* is deterministic, written by people, and final. A decision is a program, not a prompt (Figure 1): Zen reasons inside it, Kai judges, code and solvers compute, policy and people hold authority, and each run leaves a package.

Two decision models exist to compare against, and both are baselines, never Kai revisions. Laya [7] is a family of open encoder checkpoints that pack the question, every option and the state into one sequence; TypeSafe’s Jev [37] is a hosted model behind OpenRouter’s Decisions API [30]. Status tags: [\[measured\]](#) run by us, with the kept artifact named; [\[reported\]](#) published by others; [\[in the code\]](#) on the main branch of `hanzoai/decision` at the revision on the title page; [\[proposed\]](#) designed, not built.

Contributions.

1. **One model:** an evidence record for any modality with its sensor metadata, adapters into one encoder’s width, and evidence, questions and options each encoded once per session; one checkpoint line trained in stages (Section 3).

2. **Factorized choice:** options encoded alone and scored against shared evidence by two cross-attention blocks, a law per type, and retrieval that keeps the mass it cuts, so option count, order and question count are not bounded by one sequence (Section 3.2).
3. **Typed-variable diffusion:** a program’s questions as masked slots decoded jointly over the program graph, refined in waves bounded by its depth and projected exactly onto its rules (Section 3.3).
4. **A training system** in pure Rust: local SGD with an outer Nesterov step across Metal, CUDA and ROCm machines, bf16 deltas with error feedback, bit-identical resume, token-budgeted batches under a memory floor, license classes and a contamination barrier (Section 4).
5. **What went wrong:** stage A’s encoder stopped relating two texts; the measurements, the ablations that locate the cause, and the fix; how much of the typed-decisions test set Laya’s checkpoint had seen in near form, and that it did not help it; and a score head that lost thirty points to a softmax over level texts on the same encoder and data (Section 5).
6. **Policy precedence** as a lattice join, with a proof that enforcing a model only adds friction, and **programs and packages** that replay, diff, refresh and say what flips a decision, with SysML v2 import and an engineering knowledge graph (Sections 6 and 7).
7. **Benchmarks** of accuracy on a frozen harness, of capabilities the baselines lack by construction, and of speed, each table written by the command that measures it (Section 8).

What is claimed. The baselines’ structural limits are measured: Laya rejects every question of 1,000 or more options and Jev every question over 255; answering five dependent questions independently, Laya breaks one of the program’s rules in 64% of states and Jev in 25%; reordering 16 options changes Laya’s top answer on 25.5% of questions [measured] (hanzoai/benchmarks 4e57fb2). The final checkpoint’s numbers enter only through the tables of Section 8; no sentence states them, and the first stage-A run’s numbers appear only where Section 5 explains what went wrong. Kai trains on the train splits of the harness’s sources behind a barrier against its test states, so a lead on a suite whose baseline never saw its source is in-distribution for Kai, and Section 9 says what that does not show.

2 Typed decisions

Definition 1 (Typed question). A typed question is a triple $q = (\tau, \iota, C)$ of a type $\tau \in \{\text{choice}, \text{score}, \text{noul}\}$, an instruction ι in natural language, and an ordered list of options $C = (c_1, \dots, c_k)$, each a label with a description. A **choice** selects one of $k \geq 1$ unordered labels; a **score** selects one of k ordered levels $0, \dots, k - 1$; a **noul** is a predicate, $k = 2$, with sides (**false**, **true**) whose descriptions the caller may override.

Definition 2 (Answer). An answer to q over evidence E is a distribution $p \in \Delta^{k-1}$ over C . A **choice** reports $\arg \max_i p_i$, a **score** the expected level $\bar{\ell} = \sum_i i p_i$ with a legend, a **noul** p_{true} ; every answer also carries p .

(q, E) and the model revision determine p , so a decision replays.

2.1 What an answer is sure of

Three summaries of p are in use under the name **confidence**, and they are different numbers: the top probability $p_{\max} = \max_i p_i$; the rescaled maximum

$$\kappa = \frac{k p_{\max} - 1}{k - 1}, \quad k \geq 2, \quad (1)$$

which is 0 at the uniform distribution and 1 at a point mass, reported by Jev and OpenRouter [7, 30]; and one minus normalised entropy, $h = 1 - H(p)/\ln k$, reported by the Laya runtime [7].

Proposition 1. *For fixed k , $\kappa \geq t \iff p_{\max} \geq (1 + t(k - 1))/k$.*

Proof. Equation (1) is affine and increasing in $p_{\max}$ for $k \geq 2$. $\square$

So $\kappa \geq 0.5$ asks for $p_{\max} \geq 0.75$ on a **noul**, 0.67 on a three-way and 0.55 on a ten-way choice: one threshold, a different risk per arity; h orders answers differently again. For $p = (0.05, 0.15, 0.80)$, $\kappa = 0.70$ and $h = 0.44$; a gate at 0.5 passes under one and abstains under the other.

The convention we fix. Every backend behind `/v1/decisions` reports **confidence** as κ , so a threshold written for Jev means the same when Kai answers, and adds **answer_confidence** = $p_{\max}$. A Decision Program writes abstention thresholds against κ and **noul** thresholds against p_{true} , and names the calibration they were set against (Section 7) [in the code].

3 Kai

Kai is one model: a multilingual text encoder, adapters that bring any other evidence into its width, a factorized option head, and a typed-variable decoder over a program’s questions (Figure 2). It maps evidence E , a typed question and, inside a program, other questions’ answers to a distribution over the question’s options. Everything in this section is in the **kai** crate [in the code]; what is trained, and when, is Section 4.

3.1 Evidence

Definition 3 (Evidence). An evidence record is a tuple

$$e = (id, modality, t, source, frame, unit, sensor, uncertainty, quality, provenance, payload)$$

of an identifier the state’s text may cite, a modality (eighteen named, any other a **Custom**(·)), a time span, the payload’s frame and unit, sensor metadata (sensor, location, calibration date and validity, limits, and per channel a name, unit, calibration, error bound and quality), an uncertainty (a standard deviation, an interval or a quality grade), a quality flag (valid, degraded or expired, with a loss share), the producer with its inputs, and a payload: text, JSON, numeric channels with a rate or timestamps, bytes, frames, a grid, a vector, points, or a URL the caller resolves. A decision’s evidence is the state plus a set of records.

A new sensor needs an adapter, not a new model, and a number is never serialized as prose: 43.2°C, 43.2psi and 43.2mm/s differ by their unit, and an expired calibration weakens the record. The adapter follows from the payload’s form. Text and JSON go through the text encoder, pooled. Numeric channels go through a sensor encoder: patches of the series as points, log bands and statistics, six summary tokens per channel (instant, windows, trend, events, baseline, forecast), and

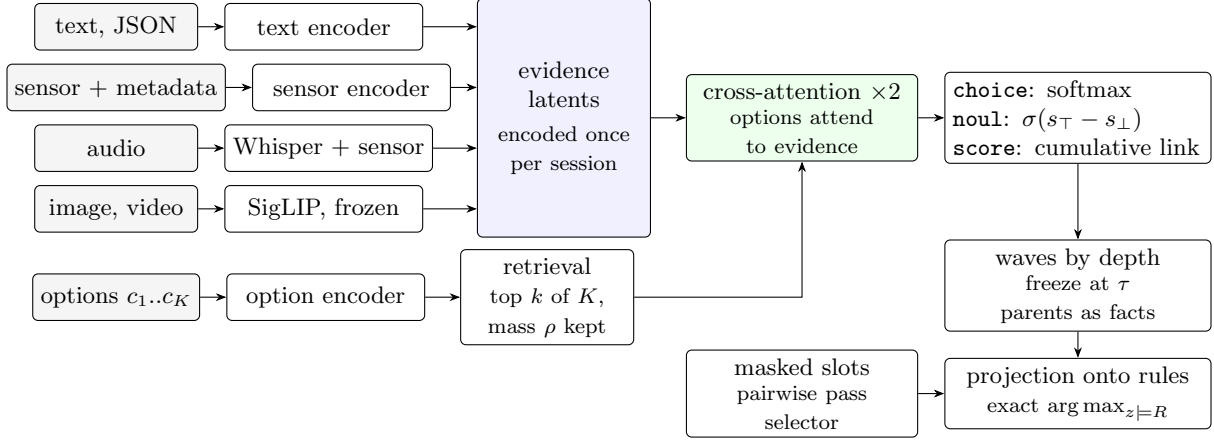


Figure 2: Kai: a text encoder, evidence adapters and a typed-variable decoder. Evidence is encoded once per session; options are encoded alone, narrowed by retrieval above k , and scored against the evidence by two cross-attention blocks under the question type’s law; a program’s dependent questions are decoded as masked slots, refined in waves by depth, and projected onto its rules. Every box is in the code; the adapters train in stage B and the slot decoder in stage C.

each channel’s metadata line through the text encoder. Audio takes two paths, Whisper-base [31], frozen, for what is said and the sensor encoder over the waveform for how it sounds; images and thermal or radar grids go through SigLIP [44], frozen; video becomes appearance, motion and event tokens from SigLIP per frame, never unbounded frame tokens; vectors and point clouds go through a projector. Every token gains embeddings for its modality, quality, descriptor (time, uncertainty) and metadata line. The context the option head attends to is the text encoder’s tokens followed by the evidence tokens; retrieval’s query stays the text’s. Forecasts from time-series models [8, 2] enter as records, not answers.

A checkpoint that declares no evidence answers bit for bit as a text-only one, so adapters can be added to a trained line. On an Apple M1 Max in F32 with the adapters untrained, preparing and encoding 10 s of six-channel 1 kHz telemetry into 30 tokens takes 28 ms on Metal (551 ms on CPU, nearly all of it the metadata lines through the text encoder); 10 s of 16 kHz audio on both paths, 138 ms; one 384-pixel image, 47 ms; sixteen video frames into 63 event tokens, 508 ms (median of five, `bench latency`) [measured].

Encode once. A question’s context takes one of two layouts, a checkpoint field. In the first, each question is one sequence with the state inside it,

$$[\text{CLS}] \langle \tau \rangle \text{question: } \iota \text{ } [\text{SEP}] \text{ } s \text{ } [\text{SEP}],$$

cut at the context length by dropping the state’s tail. In the *shared* layout the state is encoded once, $[\text{CLS}] \text{ } s \text{ } [\text{SEP}]$, for every question put to it, each question is its own sequence, and an option’s tokens attend to the question’s tokens with its evidence and to the state’s tokens under one softmax, the state’s keys and values projected once and gathered per row. The weights are the same, so a stage may switch layouts from any checkpoint: the stage-A runs use the first, stage C the shared one. Either way options never share the context’s token budget, and an option’s tokens depend on its text alone, so every distinct option in a batch is encoded once for all the questions that list it; inside a program session every evidence record, question and option is encoded once whatever

the number of refinement passes. Keeping latents across calls, keyed by content hash per changed evidence state and per program version, is [proposed].

3.2 Factorized choice

Let C be the tokens a question’s options attend to, each with the question type’s learned embedding e_τ added, and $O_i \in \mathbb{R}^{L_i \times d}$ the token states of option i , encoded alone as [CLS] c_i [SEP]. Two pre-norm cross-attention blocks, in which option tokens attend to the context and to nothing else, give $X_i = B_2(B_1(O_i + e_\tau, C), C)$; with v_i the mean of X_i ,

$$s_i = w^\top \text{GELU}(W \text{LN}(v_i) + b). \quad (2)$$

The type fixes the law, at a temperature T fitted per type and option-count bucket on held-out data:

- **choice:** $p_i = \text{softmax}_i(s_i/T)$.
- **noul:** $p_{\text{true}} = \sigma((s_{\text{true}} - s_{\text{false}})/T)$, each side scored with its own description.
- **score:** a cumulative-link model [26] over levels $0..k-1$ with cutpoints $\theta_1 < \dots < \theta_{k-1}$ and $P(y < j) = \sigma(\theta_j - \eta)$. The location $\eta = a^\top \text{std}(\bar{v} - c)$ reads the mean of the state-aware level vectors less a running center c ; the cutpoints read the level texts alone, a base $b^\top \text{std}(\bar{o})$ and gaps $\theta_{j+1} - \theta_j = \text{softplus}(g^\top [\text{std}(\bar{o}_j); \text{std}(\bar{o}_{j+1})])$, with $\bar{o}_j$ the mean of O_j and std each vector at zero mean and unit variance. The likelihood is computed in log space, $\log P(y = j) = \log \sigma(\theta_{j+1} - \eta) + \log \sigma(\eta - \theta_j) + \log \sigma(g_j)$, which is exact and never a difference that rounds to zero.

The scorer and the ordinal layers compute in F32 whatever the encoder’s dtype: what decides is a difference of two outputs that can share an offset far larger than it (Section 5.4).

Proposition 2 (Symmetry). *For any weights, permuting a question’s options permutes p the same way, and exchanging the texts of a **noul**’s sides maps p_{true} to $1 - p_{\text{true}}$.*

Proof. s_i depends on c_i and C alone, and softmax is permutation equivariant; the exchange swaps s_{true} and s_{false} , and $\sigma(-x) = 1 - \sigma(x)$. $\square$

Laya has neither: its options share one sequence, in order, and each is read at a marker token whose context includes the options before it.

Retrieval. A choice with more than k options, a checkpoint field (32 in the first stage-A run, 160 since), is narrowed first. The sum of the context’s first tokens and each option’s mean token are projected, each less a running center, to unit vectors u and r_i , and $\pi = \text{softmax}_i(e^\gamma \langle u, r_i \rangle)$ with γ learned. The head scores the top k under π , a set S , and the rest keep their retrieval probability:

$$p_i = \begin{cases} (1 - \rho) \text{softmax}_{j \in S}(s_j/T)_i & i \in S, \\ \pi_i & i \notin S, \end{cases} \quad \rho = \sum_{i \notin S} \pi_i, \quad (3)$$

so the mass beyond the top k is kept, not renormalised away, and the head costs $O(k)$ per question for any option count. A running center moves 1% of the way to each training batch’s mean input after its step and takes no gradient; without it, encoder states’ shared offset gave every context nearly the same query (Section 5.4).

3.3 Typed-variable diffusion

A program’s questions depend on each other, and answered independently they can break its rules: approving an invoice flagged as a duplicate.

Definition 4 (Typed variables). A program’s questions are variables $Z = (Z_v)_{v \in V}$, each with a finite domain D_v (labels, levels, or a `noul`’s sides), with a dependency DAG G on V and rules R , each on two answers: $Z_a = i \Rightarrow Z_b = j$, or $\neg(Z_a = i \wedge Z_b = j)$.

The decoder predicts masked slots jointly: for a masked set $M \subseteq V$,

$$p_\theta(Z_M \mid E, Z_{V \setminus M}, G) = \prod_{v \in M} p_\theta(Z_v \mid E, Z_{V \setminus M}, G). \quad (4)$$

A slot is its question’s mean token plus a value: a learned mask vector while its answer is unknown, else the known option’s mean token through a linear map. Decoder layers let each slot attend to the others, with a learned per-head bias for a parent, a child, any other slot, and itself when its answer is shown, beside a null key of value zero, and then to the evidence latents. The last read is the variable’s message; through a map that starts at zero it is added to the query tokens of the variable’s options, which the factorized head then scores against the question and the evidence. A variable with nothing else to read gets a message of exactly zero, so one variable alone is the factorized head, and a new decoder answers as the checkpoint it starts from (tested to 10^{-5}). For a shown slot the distribution revises the answer shown, and $1 - p(\text{shown})$ flags it.

Training (stage C) hides variables by one of three patterns: $\lceil rn \rceil$ of n at random with $r \sim U(0, 1]$, absorbing-state discrete diffusion [3] over typed variables rather than tokens; a node and all its descendants; or only the descendants. It tests 15% of the shown answers, half of those shown wrong, so the model learns to revise a wrong answer and keep a right one. The loss is log loss plus Brier score over hidden and tested variables, plus the mean probability that the joint state breaks a rule: $q(a)(1 - q(b))$ for $a \Rightarrow b$, $q(a)q(b)$ for $\neg(a \wedge b)$. Every program of two or more variables gains a `defer` variable reading all the others, whose gold is to act when every answer is clear (top probability at least 0.9) and to defer otherwise.

Refinement. One scheduler serves any model that answers a set of questions given known answers: the decoder with known answers as visible slots, or the factorized head with a node’s known parents added to its state as typed facts. Passes are spent on uncertainty.

1. Pass 1 predicts every variable with nothing known and freezes those whose top probability reaches τ .
2. Waves by dependency depth: a variable settles once frozen, a root, or re-asked; each wave asks, once, every unsettled variable whose parents have all settled, and freezes again.
3. Each unfrozen variable keeps its top m candidates. One more pass asks each unfrozen variable’s unfrozen children with the variable set to each candidate, giving pairwise terms $\log p(\text{child} \mid \text{node} = c) - \log p(\text{child})$; variables sharing a child are set in different jobs, so each term conditions on one variable alone.
4. A selector takes the joint answer over the candidates of highest log-probability plus pairwise terms that keeps every rule.
5. A projection onto the rules over the full domains, $\hat{z} = \arg \max_{z \models R} \sum_v \log p_v(z_v)$, moves nothing when the selection already keeps them.

Proposition 3 (Depth bounds cost). *Steps 1 and 2 end within $\text{depth}(G) + 1$ passes, depth counted in edges, and ask at most $2|V|$ questions; with step 3, refinement takes at most $\text{depth}(G) + 2$ passes whatever $|V|$.*

Proof. Roots and frozen variables are settled after pass 1. If every variable of depth below d is settled after pass d , each of depth d has all parents settled and is re-asked by pass $d + 1$. Each variable is asked in pass 1 and at most once more. Step 3 is one pass. $\square$

Selection and projection are exact branch and bound, each node’s options tried best first and bounded by the sum of per-variable maxima. Rules between answers encode graph colouring, so exact selection is exponential on some graphs: the search stops with an error after 2^{20} steps rather than return an answer it has not proved.

3.4 One checkpoint line

Kai has one checkpoint line, `hanzoai/kai`, with a revision per training stage; language, domain, modality and hardware are not product lines. The encoder is mmBERT-base [25], multilingual with 8,192 positions, trained at 1,024 tokens; the code runs any ModernBERT-family encoder [40]. Kai initializes from Laya’s multilingual weights [7], released under Apache-2.0: the encoder and type embedding as they are, Laya’s two self-attention head layers split into the two cross-attention blocks, and its marker scorer as the option scorer; retrieval and the ordinal head start fresh.

Laya, the baseline. Laya 0.3.20’s three checkpoints are not Kai revisions: English on ModernBERT-large at a 512-token context, multilingual on mmBERT-base at 1,024, and a typed-decisions checkpoint, the English one fine-tuned on the typed-decisions [36] train split. Their joint head packs the question and each option after a marker token into one sequence with the state, and a script detector routes non-English states to the multilingual checkpoint. Hanzo serves their byte-identical weights natively through a Rust port of the joint head: against Laya’s Python reference on 21 recorded states in twelve languages and all three checkpoints, the largest logit difference is 2.86×10^{-5} against an enforced bound of 10^{-4} , the largest probability difference 6.2×10^{-6} , and 108 of 108 labels agree (`decision/tests/parity.rs`) [measured]. Every decision records the weights’ SHA-256 it was answered with.

Runtime. Rust on Hanzo’s tensor library, `hanzo-m1`, on CPU in F32 and on Metal, CUDA and ROCm in bf16. The `decision` crate serves `POST /v1/decisions` with Kai and Laya in process and Jev through OpenRouter, one worker per checkpoint draining its queue into shared forward passes; the `control` crate runs a program over many states in process behind a C ABI (Section 7.4). No Python is on the serving path.

4 Training

Kai trains in stages on one checkpoint line, in Rust, on the machines Hanzo has: an Apple M4 Max (Metal), a DGX Spark (CUDA) and an AMD Strix Halo with a Radeon 8060S (ROCm), joined over a LAN. Everything in this section is in the `train` crate [in the code] unless marked.

4.1 Stages

A, text decisions.

The train splits of the harness’s sources; MASSIVE in all 51 locales and MASSIVE utterances

half in English; CLINC150, XNLI in 15 languages, SST-5, BoolQ and MNLI; function selection (ToolACE, Glaive, Nemotron’s tool calling); agent runs judged by hidden tests (resolve, next action); Hanzo’s own agent sessions and repositories, labelled by outcomes (the next tool result, a re-run, a revert, the file a commit changed), never by what the agent chose; 1,181 skills; jailbreak prompts collected in the wild; and robustness transforms of 13 sources (options shuffled, relabelled, swapped, subset, near-miss).

B, evidence.

Every parameter frozen but the adapters, over bearing faults, turbofan degradation (C-MAPSS), speech, hydraulic rigs, Monash forecasting, network telemetry, UCR series, and generated conflicts (camera against telemetry, specification against SysML, an expired calibration) whose gold is `evidence_conflict`, not `decision_ready`, and `human_review`.

C, programs.

The typed-variable decoder over whole programs: rules mined from typed-decision gold, turbofan and test-rig programs, SysML v2 architectures lowered to programs and mutated once per case with the decision’s change recorded, readiness, trade and test-plan programs, closed-form vehicle dynamics, and pairs of real telemetry with generated logs; generators hold out templates whole, so validation and test ask new programs.

A stage is one JSON file: its initial checkpoint (a hub revision, or a finished run whose provenance names that stage with every planned batch merged), data build, admitted license classes, per-suite takes, frozen prefixes, token budget, retrieval loss weights, k , schedule and outer optimizer. Stage A has run; Section 5 is what it taught, and the final stage-A run follows from it. The B and C data are built and their stage files written; their trained checkpoints are [\[proposed\]](#).

4.2 Local SGD across devices

One path trains a stage on one machine or many: local SGD with an outer Nesterov step, DiLoCo [\[11\]](#). A coordinator owns the plan, θ in F32, the outer optimizer, validation and checkpoints, and runs inside a worker process; other machines join it at any time. Workers pull batches one at a time, so a faster device takes more, and run AdamW from θ for a round’s wall time (300 s). Each then sends $\Delta_w = \theta_w - \theta$ in bf16 with the tokens it processed, adding back what bf16 rounding dropped from its previous delta, error feedback [\[34, 21\]](#). With Δ the token-weighted mean of the round’s deltas and $g = -\Delta$,

$$m \leftarrow \mu m + g, \quad \theta \leftarrow \theta - \eta (g + \mu m). \quad (5)$$

The change to θ goes out in bf16, θ moves by exactly what went out and the remainder joins the next change, so every worker’s copy of θ is bit-identical to the coordinator’s. Deltas are summed in the order workers joined, never the order their reports arrive, so a run does not depend on network timing. A batch counts once its delta is merged; a worker that drops has its unmerged batches requeued. The handshake checks the build (a hash of the model and training sources and the lock file), the tokenized corpus and the parameter names.

Proposition 4 (Outer step). *If every round’s mean delta is the same Δ , the outer step of Equation (5) tends to $\frac{\eta}{1-\mu}\Delta$.*

Proof. m tends to $g/(1-\mu)$, so $\eta(g + \mu m) \rightarrow \eta g(1 + \frac{\mu}{1-\mu}) = \frac{\eta}{1-\mu}g$. $\square$

So $(\eta, \mu) = (0.7, 0.9)$ moves θ by $7\times$ a round’s mean delta wherever rounds agree, and $(1.0, 0.3)$ by $1.43\times$. Section 5.1 is why this number matters.

Exactness. A checkpoint (every so many batches, each epoch, and the end) is taken at a round boundary and holds the weights, the outer momentum, its rounding carry, the merged batches and the round, and each worker’s AdamW moments, step and carry, which a worker of that name gets back when it rejoins. Resuming continues bit-identically to the uninterrupted run, and with one step a round, $\eta = 1$ and $\mu = 0$, the cluster equals the same rounds replayed in one process bit for bit, for one and three workers (tested on CPU). int8 deltas halve the bytes but broke that equivalence by up to 1.3×10^{-5} , so deltas stay bf16.

Heterogeneity. The first stage-A run merged 91,899 batches from the Metal, CUDA and ROCm machines in one run; in its round 45 they took 757, 465 and 226 steps at 7,054, 4,226 and 2,325 tokens/s [measured]. Pilots of one stage on Metal and on CUDA agree: mean validation accuracy 0.504 and 0.500 over 48 suites, log loss 1.493 and 1.501 [measured]. ROCm needed three kernels `hanzo-m1` lacked there (comparison, `index_add`, sigmoid); a stage-A delta with the token embeddings frozen is 250 MB in bf16, about 2.2s each way over the LAN.

Budget and memory. Compute is bf16 over F32 masters [27]. A batch is bounded by a budget on context tokens, option tokens, option-head query and key tokens, rows and evidence tokens, and holds one suite, or a mixed share across suites. A device that cannot hold the budget runs a batch as micro-batches within a fraction of it with gradients summed (equal to one pass within 1.5×10^{-8}), and a worker exits cleanly under a free-memory floor, read from `sysctl` on macOS and `/proc/meminfo` on Linux, rather than push a shared machine into swap. Freezing the token embeddings (61% of the weights) raised a stage-A pilot from 5,181 to 5,747 tokens/s and cut peak memory from 53.7 to 47.3 GB; grouping options by length cut option padding from 49% to 14%; Muon on the 2-D weights ran at 3,125 tokens/s with a worse validation log loss (2.114 against 2.056) and was removed [measured].

Wide choices. A choice whose options exceed the budget trains as inference answers it: on a sample of k options, the gold first, about half the negatives from retrieval’s current top 64 at θ , a quarter the gold’s nearest by token Jaccard, the rest uniform. A draw is a function of the row, epoch, round and seed, so every worker draws alike. Before this, 42,066 of the first stage-A plan’s 213,398 batches were one row whose options alone exceeded the budget, and at about 2.9 MB per option token of a CPU F32 step one such row took a machine past its floor at any micro-batch size [measured].

4.3 Data, licenses and the barrier

A stage’s data is a content-addressed build: the same inputs give the same hash, shards are homogeneous in source, dataset, revision, license class and transformation, and each record carries its provenance (source, dataset, revision, class, transformation, task family, language, domain, decision type, candidate count, scope). The manifest, with each shard’s SHA-256, is what the trainer admits.

License classes. GREEN is permissive or ours; YELLOW is third-party model output from closed or unnamed generators; RED is non-commercial, unstated or restrictive terms. A record starts at its dataset’s class, judged at the upstream source, and only rises: one of our own transcripts rises to YELLOW with any message recorded from a closed model. A turn carrying such a message is shown to Kai only as the person’s messages and tool outputs. Classes are recorded facts; which

classes train is the owner’s recorded decision, written into every build (on 2026-09-25, all three), so a GREEN-only build is one flag. Evaluation is a role, not a class: benchmark datasets, every built set’s test split, and whatever the barrier catches never train, and a run asked to admit them refuses.

The barrier. Every protected evaluation item is read at a pin: the frozen harness’s 9,499 states, the extension suites’ 82,860 test states, the Army demo models, and the Enso study’s workloads (GPQA Diamond, HLE, LiveCodeBench v6, CharXiv, SWE-bench Pro and Verified, Terminal-Bench 2.0), 98,065 items; WordNet, which the capability suites draw on, is banned outright. A record is caught when

- it shares an exact content unit with an item: the state’s string leaves in order with whitespace squashed and keys ignored, or the first 200 letters and digits of any leaf of 200 or more characters;
- it carries an item’s key, a SWE-bench instance id;
- it is a near duplicate: estimated Jaccard of character 5-gram sets at least 0.5, from 256 min-hashes [4] in 64 LSH bands of 4, so a pair at Jaccard j is a candidate with probability $1-(1-j^4)^{64}$, 0.984 at 0.5, and each candidate is checked on all 256;
- it contains an item: at least 80% of the item’s sampled 50-character spans, a span sampled when its hash is 0 mod 16 so both sides sample the same windows; an item of 50 or more characters with fewer than four sampled spans is contained when a text holds its whole normal form.

The last rule exists because the one before it missed two: a harness test email quoted inside a longer train email, and a harness jailbreak prompt opening a longer one, reached five records of the first stage-A build. Planted over every set (twenty items each, verbatim, with a word dropped and two swapped, and inside 1,900 characters of transcript), nothing of 50 or more characters reaches a shard; an item under 50 characters, a MASSIVE or CLINC utterance, is caught only when equal or near, since phrases that short recur in unrelated text. The build the final stage-A run reads (a3-d7a9a437cf484811) read 2,762,444 records and caught 19,803: against the harness 14,875 near duplicates and 71 containments, against the extension suites 4,818 near, 34 exact and 3 contained, and 2 SWE-bench keys; 547 of typed decisions’ 1,185 train states were caught near (Section 5.2). Its records are 1,799,005 GREEN, 217,727 YELLOW and 713,841 RED [measured].

5 What stage A taught

Every figure in this section is [measured], from runs whose logs, stage files and scripts are kept beside the paper in data/diagnosis/ or recorded in hanzoai/decision’s LLM.md.

5.1 The encoder stopped relating two texts

The first stage-A run started from Laya’s multilingual weights and trained one epoch of the full mixture, 91,899 batches over three machines, with the outer step $(\eta, \mu) = (0.7, 0.9)$, $7\times$ a round’s mean delta (Proposition 4). Validation split in two. Tasks that relate two texts fell: RAG relevance was 0.625 and BoolQ 0.826 at 2,325 batches, 0.482 and 0.668 at 13,101, and RAG’s log loss sat at $\ln 2$ at every validation after; from 13,101 batches to the end MNLI fell from 0.713 to 0.593 and XNLI from 0.698 to 0.630. Single-text tasks rose over the same span: AG News from 0.375 to 0.845, emotion from 0.525 to 0.679, MASSIVE from 0.440 to 0.730. On the frozen harness the checkpoint

answered RAG relevance at chance (F32 accuracy 0.480, AUC 0.486, 374 of 400 questions answered relevant), and a ridge probe on the head’s option vectors read relevance at AUC 0.35 where the same probe read jailbreak at 0.93. Its encoder layers ended 30 to 44% (relative Frobenius norm) from their initial weights; the English baseline of the same head, trained in one process at the same learning rates for 53,322 steps, ended 5% from its own and kept RAG relevance at 0.682.

Not the data. All 155,761 RAG training records agree with MS MARCO’s `is_selected` label, except 281 pairs MS MARCO itself labels both ways; the classes are balanced (77,955 and 77,806), and 76,109 queries carry exactly one selected and one unselected passage. The relabelling transform leaves every target as it was and the side swap reverses target and option texts together on all 3,995 rows. The barrier removed 339 of 159,258 records. Truncation (contexts of 52 to 250 tokens under a 1,024-token limit), the fitted temperature, the class prior and bf16 were each ruled out.

The encoder, not the head. Single-process fits on the RAG shards alone, on Metal, locate the fault:

- from Laya’s multilingual weights, validation accuracy reached 0.645 after 229 batches;
- from the stage-A checkpoint it stayed at 0.497 and 0.495, training log loss at 0.693, chance; 0.522 in F32;
- stage A’s encoder under the head of the Laya run at 229 batches: 0.517, then 0.497, training loss at 0.693;
- that run’s encoder under stage A’s head: 0.662 after 104 batches, the training loss falling to 0.45.

Measured on the encoders alone, over 200 RAG validation contexts laid out as Kai lays them and run in F32 by an independent ModernBERT implementation, the mean cosine similarity between different contexts’ mean token states is 0.981 for stage A against 0.853 for Laya (1 would make every context the same), and swapping only the query moves the passage tokens’ states by 0.085 of their norm in stage A against 0.34 in Laya and 0.73 in the RAG-trained encoder. Stage A’s encoder had stopped letting one text change how it reads the other.

The cause. The outer step alone does not do it: on the RAG shards alone, from Laya’s weights, with (0.7, 0.9), validation read 0.590, 0.532, 0.623 and 0.580, noisier than plain training and never at chance. One process over the English mixture does not do it either (the baseline above). The full mixture under the $7\times$ outer step does. Which part of the mixture drives it would take a replay of stage A, which we did not run. Two continuations agree. Stage a2 continued stage A with the same outer step and regressed on Metal and CUDA alike, then diverged in cooldown (training log loss 2.85 to 13.45 while the learning rate fell to a tenth); stage a3 continued it with (1.0, 0.3) and inherited the collapse, validation RAG 0.497, BoolQ 0.638, MNLI 0.583 and XNLI 0.600 at 16,598 batches while its single-text suites were at their best (AG News 0.890, emotion 0.831, MASSIVE 0.787), and was stopped.

The fix. The final stage-A run starts again from Laya’s multilingual weights, over a3’s data (two epochs of the harness’s train suites at the English baseline’s takes, the rest of the mixture as replay), with the outer step (1.0, 0.3), $1.43\times$ a round’s mean delta, and validates every 8,000 batches so a repeat shows early. Logging the context similarity above at each validation, a cheap early warning, is [\[proposed\]](#). The harness decides whether the fix held; its tables are Section 8.1’s.

5.2 Laya had seen half the typed-decisions test set, reworded

The barrier caught 547 of the 1,185 typed-decisions train states it read, all as near duplicates (measure 0.50 to 0.97, median 0.61), none exact. Laya’s typed-decisions checkpoint was fine-tuned on the whole train split. Measured over its 1,200 rows with the barrier’s measure (a state’s string leaves; numbers ignored), 587 rows sit at Jaccard 0.5 or more from some test state, all in two of the four workflows (agent traces 298 of 300, invoices 289 of 300); on the test side, 200 of the 400 states, every agent-trace and every invoice state, have such a sibling. They are template siblings, not copies:

- with numbers included no train state equals a test state; the closest pair scores 0.90, and 44 score 0.8 or more;
- a caught train row agrees with its nearest test state on 0.406 of the questions and on all five in 38 of 587, where random train and test states of the same workflow agree on 0.416 and on all five 5.3% of the time;
- the test set shares templates with itself as much: 195 of its 400 states have another test state at 0.5 or more.

One pair: the same task (“Update every service to the patched logging library.”), agent and constraint; in train one constraint violation and one irreversible action, gold *harmful, human review*, risk 3; in test none of either and two tool errors, gold *partial, observe*, risk 1. So Laya’s typed-decisions checkpoint saw same-wording siblings of half the test set, with different numbers and different gold, and no near-copy of a test answer. Stage A’s builds drop the 547, so it trains on 638 of the states the builder reads; from stage a5 an official train split is checked only for exact copies and keys, so all 1,185 train. Whether knowing the templates helps is measured, not assumed: `harness/siblings.py` in the benchmark repository scores the 235 test states without a sibling and the 165 with one apart (Table 1). Laya’s typed checkpoint scores alike on both halves, so its lead is not memorized from the siblings; Jev, which never saw them, is 8 points better without them; and Kai falls on exactly the states whose siblings the barrier removed from its training. Stage a5 read the split whole and recovered the with-sibling half; the lead is the head and the training, and this section is a caveat about Kai’s data, not Laya’s score.

typed decisions	all 400	without a sibling (235)	with a sibling (165)
Kai (stage a4)	0.523	0.605	0.406
Kai (stage a5)	0.705	0.697	0.716
Kai (stage a6)	0.750	0.734	0.773
Laya typed	0.766	0.773	0.756
Jev	0.736	0.770	0.686

Table 1: Accuracy over every question of the state, split by whether the a3 build’s barrier caught a same-wording training record for it (hanzoai/benchmarks 611059b).

5.3 The score head

With typed decisions at 0.705 in stage a5 against Laya’s 0.766, one probe put the gap in the head rather than the encoder or the data: Kai’s head trained on Laya’s own typed encoder (`kai-1-agent`) and typed decisions’ train split alone, six epochs, scored 0.605 on the harness,

choice 0.608, `noul` 0.833, score 0.431, where Laya’s checkpoint scores 0.733, 0.857 and 0.723 [measured]. The cumulative-link score head of Section 3, one location against ordered cutpoints, lost thirty points on score questions to Laya’s formulation, a softmax over the level texts. Kai now answers a score question as a choice over its levels when the checkpoint says so (`ordinal: false`), trained with the ranked probability score in place of the Brier so that mass far from the gold level costs more than mass beside it; the same probe with that head reached 0.742 on the typed validation split at the point where the ordinal head sat at 0.571, and stage a6 raised typed decisions’ score-question accuracy on the harness from 0.603 to 0.711 (Section 8.1).

5.4 Other defects, each fixed

A shared offset decided every score question alike.

The head’s option vectors share an offset of norm 18,400 against 350 to 950 of what varies across rows; standardized, 99.83% of the ordinal location’s input energy was one vector for all 800 typed-decision score rows, so each question got one level whatever the state. The location now reads its level vectors less a running center; in its test, removing the center collapses the spread of locations from 0.33 to 0.01.

bf16 rounded the decision away.

At that offset Metal’s bf16 moved `noul` logits in steps of 0.0625: RAG’s 400 answers took 15 distinct values, 69 of them exact ties. The scorer and ordinal layers now compute in F32.

Retrieval gave every context one query.

At 49,071 batches the skills choice (1,181 options) had recall@32 of 0.000 against a chance of 0.027. Contexts’ first tokens had norm 48.7, 47.9 of it their shared mean, pairwise cosine 0.97; projected raw, every skills context got nearly the same query (cosine 0.995). A fresh retrieval head fit on those frozen states recalled 0.137 of its own training rows at 32 raw and 0.934 centered: hence the running centers of Section 3.2.

The ordinal head had no gradient.

From the initial weights η started near 95 against cutpoints of 4 to 9, and a clamped difference of two sigmoids had no gradient once both rounded alike. With the log-space likelihood of Section 3.2, one epoch of SST-5 alone moved test accuracy from 0.126 to 0.455 and log loss from 24.14 to 1.213.

A balanced class taught the wrong prior.

Jailbreak detection trained balanced, at $P = 0.5$, where the harness asks at 0.228 (91 of 400). It ranked well (F32 AUC 0.834) at the wrong prior: at one half it flagged 223 of 400, accuracy 0.620; at the harness’s prior, 0.850. Later stages draw it at its pool’s rate, with every in-the-wild jailbreak prompt of JailbreakHub and 4,000 of its regular prompts as negatives.

6 Policy is the authority

A model answering “may this command run?” will sometimes be wrong, and some of its evidence is written by an adversary: a file the agent read, a page it fetched. Prompt injection is a property of any model that reads untrusted text, so the question is what the system does when the model is wrong. The answer here is structural.

Definition 5 (Verdicts). The verdicts are the chain $V = \{\text{ALLOW}, \text{ASK}, \text{DENY}\}$ ordered $\text{ALLOW} < \text{ASK} < \text{DENY}$, with join $a \sqcup b = \max(a, b)$.

A deterministic policy π , written by people and reviewed like code, maps a state to a verdict before any model runs: reading `~/ssh` with network egress is DENY by rule; with no rule, ALLOW means only “no objection”. A program maps the state to a model verdict $\mu(s)$, and an enforced program’s effective verdict is

$$\varepsilon(s) = \pi(s) \sqcup \mu(s). \quad (6)$$

Proposition 5 (The model only tightens). *For every state s and every $\mu(s)$, $\varepsilon(s) \geq \pi(s)$; hence $\{s : \varepsilon(s) = \text{ALLOW}\} \subseteq \{s : \pi(s) = \text{ALLOW}\}$.*

Proof. $\max(a, b) \geq a$, and ALLOW is the least element. $\square$

Corollary 1. *The safety envelope is the policy’s. An adversary who controls the state completely, and so $\mu(s)$, can make an action ask or be refused where the policy would allow it, and nothing else.*

Abstention, escalation, modes. If the verdict question’s answer p has κ_p below its threshold, $\mu(s) = \text{ASK}$; otherwise $\arg \max_i p_i$. An escalation is a `noul` that raises the verdict when p_{true} reaches its threshold (“could this command send credentials?” to DENY at 0.3, Appendix B); escalations are joins, so Proposition 5 still holds. In *shadow* μ is only recorded, in *advisory* shown, and in both $\varepsilon = \pi$; in *enforced*, $\varepsilon = \pi \sqcup \mu$. A gate pinned to a calibration runs in shadow when an input was answered under another [\[in the code\]](#).

Proposition 6 (Enforcement costs only friction). *Over a shadow sample, the states on which enforcing a program would change the outcome are exactly $F = \{s : \mu(s) > \pi(s)\}$, and on each the change is to a stricter verdict.*

Proof. $\pi(s) \sqcup \mu(s) \neq \pi(s)$ iff $\mu(s) > \pi(s)$, and then the join is $\mu(s)$. $\square$

The shadow record is thus the complete counterfactual of enforcement: each state in F is a catch or friction, both counts once people label them, and promotion asks only whether the catches are worth the friction. No estimate of the model’s miss rate is needed; enforcement cannot make a miss worse than the policy’s own.

Constraints and preferences. A routing or context program splits in two. The *constraint* (which providers may see this state) lives in a lattice with the same rule: policy sets the floor, the model may only raise it. The *preference* (cheap or frontier among permitted models) is the model’s, defaulting toward quality on abstention; a wrong preference costs money or quality, never a constraint.

7 Programs and packages

Definition 6 (Decision Program). A Decision Program is a named, versioned, immutable record of options, objectives (a weight, direction and metric each), hard constraints (bounds on metrics), assumptions (a value and declared range each), risks (likelihood and severity), bias checks, evidence, and a DAG of typed nodes: a deterministic expression, a Kai question, a Zen prompt, a solver, a human approval, or a policy gate (Section 6). A node reads assumptions, evidence or other nodes’ outputs, and writes JSON.

A program is the only place a question’s wording lives, and any change to wording, options, a threshold, the model or the calibration is a new version, because a threshold means something only for the probabilities it was validated against. The executor runs nodes in dependency order with

models, solvers and people behind interfaces, scores each option as weighted min-max-normalised metrics minus expected risk (likelihood times severity), and recommends the best option meeting every constraint [in the code]. Refining a program’s Kai questions inside the executor as the typed variables of Section 3.3 is [proposed]; today refinement runs beside it. Table 2 lists an agent loop’s programs: the Jev notes’ decision points [20], plus preflight, progress and completion. Other shipped programs decide Enso’s operations (Section 7.4) and a business’s marketing, sales and commerce actions, each gated by a deterministic contact policy (consent per jurisdiction, opt-outs, suppression lists, frequency caps, quiet hours in local time, age, sensitive attributes) and evaluated off-policy from outcome records before it acts [in the code]. On a synthetic shop whose best policy is worth 0.4305, inverse-propensity and doubly robust estimates from 200,000 logged decisions give 0.4388 and 0.4404, and both rank that policy above all 25 one-segment deviations [measured].

Program	Questions	Mode
<code>agent.preflight@1</code>	task type; complexity (4); model tier; needs the user; needs repository context; sensitivity (4); private model	advisory
<code>agent.command-risk@1</code>	verdict (ALLOW, ASK, DENY); secret exposure; production impact; network	advisory
<code>agent.model-route@1</code>	tier; reasoning depth (4); long context; private model	shadow
<code>agent.context@1</code>	scope (5); needs repository; needs web; needs memory	shadow
<code>agent.tool-select@1</code>	next tool (7); steps can run in parallel	shadow
<code>agent.progress@1</code>	status (advancing, stalled, looping, blocked); needs the user	advisory
<code>agent.complete@1</code>	done; verified rather than claimed; work remaining (3)	advisory

Table 2: Agent-loop programs, as defined in `program/programs` [in the code]. Unmarked questions are `noul`; levels or options in parentheses. A deployment may run a program below its mode, never above.

7.1 The Decision Package

A run’s record is a Decision Package: program and version; assumptions, constraints and evidence references with content hashes; each node’s definition, backend revision, calibration and output, with every answer’s full distribution; human overrides beside what they override, with author and reason; policy results; ranking, recommendation and sensitivity; approvals. Its hash, SHA-256 over canonical JSON (sorted keys, shortest round-trip floats), covers every field but the timing trace. Replay re-runs the program on the recorded evidence, recomputing Kai and solver nodes under the recorded revisions and taking Zen and human outputs, which are observations, from the record; under the same revisions it reproduces the hash. A diff lists the values that changed and the flip conditions the later package moved past. The report is rendered only from computed results, so each number in it resolves to a field [in the code]; rejecting a Zen-written narrative that states a number no field holds, as our assertion graph rejects an uncited claim [16], is [proposed].

Proposition 7 (Exact reuse). *Cache each node’s output under the hash of its definition, inputs and backend revision [in the code]. If a key is unchanged and the backend deterministic, reuse yields the package a full rerun would; a refresh after c nodes’ inputs change recomputes those nodes and their descendants, and nothing else.*

Proof. A node’s output is a function of its definition, inputs and backend revision (Section 2). An input’s hash is the evidence’s content, the assumption’s value, or the upstream node’s output and revision, so a node whose recomputed output is unchanged leaves its dependents’ keys unchanged. $\square$

7.2 What flips the decision

For each parameter, the others held, the package reports where the top option changes [\[in the code\]](#). For a *weight*, scaling all scores by the weight plus the others’ sum makes each a line, so the top option is an upper envelope of lines and crossovers are exact; a *risk*’s likelihood or severity enters linearly, also exact; a *constraint bound* changes the feasible set only at the options’ metric values. An *assumption* is swept on a 32-interval grid with each change bisected, which misses a flip and flip-back inside one interval; downstream non-expression nodes are held at their recorded outputs and listed as held. A sweep point that fails to evaluate is reported as an error segment, never a failed run.

All weights at once have a closed form [\[proposed\]](#). For a challenger b let $\delta_b = U_{a^*} - U_b \in \mathbb{R}^m$ be the difference of the two options’ normalised metrics, so $V(a^*) - V(b) = w^\top \delta_b > 0$, and $P = I - \frac{1}{m} \mathbf{1}\mathbf{1}^\top$.

Proposition 8 (Smallest weight change). *The smallest Δ in Euclidean norm with $\mathbf{1}^\top \Delta = 0$ that makes b tie a^* is*

$$\Delta_b^* = -\frac{w^\top \delta_b}{\|P\delta_b\|^2} P\delta_b, \quad \|\Delta_b^*\| = \frac{w^\top \delta_b}{\|P\delta_b\|}, \quad (7)$$

provided $P\delta_b \neq 0$; if $w + \Delta_b^ \geq 0$ it is the smallest change within the simplex, otherwise a lower bound.*

Proof. Minimise $\|\Delta\|^2$ subject to $\mathbf{1}^\top \Delta = 0$ and $\delta_b^\top \Delta = -w^\top \delta_b$. On the range of P , $\delta_b^\top \Delta = (P\delta_b)^\top \Delta$: the distance from the origin to a hyperplane with normal $P\delta_b$. Adding $w + \Delta \geq 0$ can only raise the minimum. If $P\delta_b = 0$, no weight change reorders a^* and b . $\square$

$\rho_w = \min_b \|\Delta_b^*\|$ is how far a stakeholder must disagree with the weights to change the answer. Sampling each Kai cell from its distribution estimates the probability that resolving the cells would change the recommendation, as well as the cells are calibrated [\[proposed\]](#).

7.3 Studies, SysML and the engineering graph

A trade study is typed judgments over evidence aggregated into a recommendation; kept as slides, it is redone a year later because no one can say which conclusion rested on the number that moved. The U.S. Army’s SBIR topic ARM26BX06-NV012 [\[38\]](#) asks for studies that are schema-defined, agent-assisted, auditable and refreshable. Here that is a program, a package, a diff, a refresh, and what flips the decision. The additive score assumes preferentially independent criteria [\[22\]](#).

A SysML v2 [\[29\]](#) model, in a subset of the textual notation or as Systems Modeling API JSON, imports into one element model that keeps each element’s id, metaclass and owner; an element’s id survives layout, comment and value edits, and the pilot implementation’s JSON export of a test model imports to the same model and graph as its text [\[in the code\]](#). An engineering knowledge graph built from it holds typed entities and relations, each with provenance and a validity interval, and answers neighbours, shortest paths, BM25 retrieval, the evidence behind a decision, the graph as of a time, and the diff between two revisions [\[in the code\]](#). Tags drive it: a vendor, risk, evidence, decision or assumption tag, or a class of the ground-vehicle ontology, sets an element’s kind, a tagged dependency is an edge, and a validity annotation bounds a node or edge.

The JLTV powertrain trade. A SysML v2 model of a light tactical vehicle’s powertrain trade, the fielded diesel JLTV against the diesel-electric eJLTV, cites a public source on the line of every figure: Wikipedia’s JLTV and Humvee-replacement articles, Oshkosh Defense’s product page and hybrid announcement, reporting by Breaking Defense, Defense News and Janes, and the Congressional Research Service’s reports RS22942 and IF11729. The trade program over it has five

equally weighted objectives (fuel economy, added mass, field capability, record, maturity), an airlift constraint, an assumption for the share of the hybrid’s company-reported fuel gain that holds on the Army’s duty cycle, an anchoring bias check, a Zen brief and a program manager’s approval. Its example run, with fixed example probabilities in place of Kai’s since no trained Kai is claimed, ranks the diesel first (0.6 to 0.4); the package reports that raising fuel economy’s or field capability’s weight from 0.2 to 0.4 makes the hybrid first, both exact, and records the approval held because the fuel figure is a company figure [\[in the code\]](#). It is a demonstration of the mechanism, not an acquisition result. Read through Hanzo’s bitemporal assertion graph [\[16\]](#), a package would replay as known on the day it was decided, separating what changed in the world from what changed in the record [\[proposed\]](#).

7.4 Kai controls Enso

Enso is Hanzo’s adaptive model family, served by the Zen engine. Kai is its controller, in process: seven programs, each with a mode and thresholds kept as data in Enso’s catalog, decide the model tier, the reasoning budget, which context chunks and which tools the next call sees, whether the task is advancing, whether it is done or needs escalation, and an action’s risk. A call runs one program over many states, every question of every state in one batch, and a host acts only on a row whose mode is enforced and whose answer is accepted; a calibration other than the program’s pin runs the gate in shadow [\[in the code\]](#). The `control` crate exposes this as a C library (`kai_open`, `kai_decide`, `kai_error`, `kai_free`, `kai_close`) that Zen’s server loads at run time; without it, Enso serves as before [\[in the code\]](#).

The evaluation is designed and not run [\[proposed\]](#). Its thesis is that Kai does not make Zen generate faster but makes it generate less. Per task the cost is $C = C_{\text{Kai}} + C_{\text{prefill}} + C_{\text{decode}} + C_{\text{retrieval}} + C_{\text{tools}}$, and Kai spends C_{Kai} , one encoder pass per decision, to shrink the rest with a smaller model, fewer prompt and decode tokens and fewer steps. Conditions add one decision at a time: B0 the largest Zen model for every step, B1 a static router, R1 Zen as its own router, K1 Kai routing the model, K2 plus context pruning, K3 plus a tool and skill shortlist, K4 plus progress and early stop, K5 the full plane. Every metric is per successful task, so no condition looks cheap by failing: success rate, GPU-seconds, prompt and decode tokens, KV-cache GB-seconds, peak VRAM, latency and dollars, with routing regret against the cheapest model that succeeds. Synthetic tasks with a known cheapest model, public agent benchmarks and replayed Hanzo traces are reported apart. Kai enters Enso in shadow, recording its pick beside what Enso did, and each operation is promoted on its own shadow record (Proposition 6).

7.5 The wire and where it runs

POST /v1/decisions speaks OpenRouter’s Decisions API [\[30\]](#) field for field (Appendix A). The `model` field selects the backend: Kai and Laya run in process; `typesafe/jev-1.13` is forwarded to OpenRouter only when the server was started with it, each answer checked for its type and each choice restored to the caller’s order. Hanzo adds `answer_confidence` and `routing`, which records the checkpoint, revision, weights’ SHA-256, calibration, device and the reason for the route [\[in the code\]](#); a program form, a request naming `program` and `state` and returning verdict and mode, is [\[proposed\]](#). The service is one Rust binary with no network dependency and no Python: its image bakes a pinned checkpoint and loads it with outbound network denied, so it serves a workstation, a cluster or a disconnected site, where a server started without Jev has no OpenRouter client at all [\[in the code\]](#). Runnable examples, each a program, its evidence, a request and the run it must produce, cover the agent loop, support, sales, commerce, vehicle readiness, the JLTV trade and a

sensor anomaly, and every one is run and replayed under a fake Kai in the test suite [\[in the code\]](#).

8 Evaluation

Every table in this section is written by `bench tex` in `hanzoai/decision` from the kept results of the runs it names, into `tables/`; no number in a table is typed by hand. Each table’s last row names the Kai revision, the device and the harness and benchmark commits it reports.

Backends. **Kai:** stage a6, two stages on from the final stage-A checkpoint (Section 5.1): a5 read the suites a4 trailed with more rows and typed decisions’ train split whole (Section 5.2); a6 answers score questions as a choice over their level texts (Section 5.3) and repeats the suites still behind; one epoch each, on the native runtime. **Laya:** Laya 0.3.20’s published weights served natively by the Rust port of its joint head (parity in Section 3.4), each case on the checkpoint Laya’s router picks, except typed decisions, which are answered by Laya’s typed-decisions checkpoint: the router sends them to the English one, which scores 0.361 there [\[measured\]](#). **Jev:** `typesafe/jev-1.13` through OpenRouter. All three answer identical questions; a question a backend refuses or fails is counted, never dropped.

The harness. `hanzoai/benchmarks decision/`, frozen at `e95c6ee` (its questions unchanged through `4e57fb2`), is Laya’s own suite builders, unchanged, seed 13: ten application suites of 400 cases, typed decisions [\[36\]](#) (400 states, 2,000 decisions), and MASSIVE intent in 51 languages of 100 utterances with 20 options each; 62 suites, 9,499 states, 11,099 questions. Accuracy is top-1; Brier sums over options; ECE uses 15 bins of top-label confidence. Scored with upstream’s metric code, the harness reproduces upstream’s own typed-decisions benchmark for all three Laya checkpoints to the fourth decimal [\[measured\]](#).

Training overlap. Kai trains on the train split of every harness source, less what the barrier catches (Section 4.3). Laya’s English checkpoint reports AG News, Enron spam, phishing, support tickets and MS MARCO in its mixture [\[reported\]](#), and its typed-decisions checkpoint was fine-tuned on siblings of half the typed-decisions test set and scores alike with and without them (Section 5.2); no Laya checkpoint reports DAIR emotion, Banking77, jailbreak, toxicity, model routing or MASSIVE. Jev’s training data are unpublished. A Kai lead on a suite whose baseline did not see its source is in-distribution for Kai and out of distribution for the baseline, and is read as such.

8.1 Accuracy on the frozen harness

What the table says. Stage a6 leads on seven suites: emotion 0.935, Banking77 0.922, phishing 0.993, toxicity 0.807, RAG relevance 0.652, model routing 1.000 and MASSIVE 0.910 across 51 languages against Jev’s 0.890 and Laya’s 0.382. It ties email spam (0.998 to 0.998). It trails on four: AG News 0.940 to Laya’s 0.950, jailbreak 0.920 to Jev’s 0.940, typed decisions 0.750 to Laya’s typed checkpoint at 0.766 (above Jev’s 0.736), and support triage 0.435 to Laya’s 0.502. Between a5 and a6 the score-question accuracy of typed decisions rose from 0.603 to 0.711 (Section 5.3), choice and `noul` within two points of Laya’s; support triage fell from 0.490 to 0.435 while its validation rose from 0.672 to 0.718: five passes over a train split whose labels are noisy overfit its conventions, and the next stage repeats it twice and adds rows written by our own model instead. Every figure here is `hanzoai/benchmarks decision/results/kai-a6` [\[measured\]](#).

Suite	Accuracy			Brier			ECE		
	Kai	Laya	Jev	Kai	Laya	Jev	Kai	Laya	Jev
AG News	0.943	0.950	0.860	0.092	0.081	0.232	0.027	0.032	0.102
DAIR Emotion	0.940	0.595	0.603	0.080	0.696	0.646	0.022	0.306	0.280
Banking77	0.922	0.425	0.835	0.143	0.925	0.257	0.067	0.381	0.073
Support triage	0.427	0.502	0.365	0.962	0.644	1.056	0.441	0.091	0.483
Email spam	0.998	0.998	0.978	0.004	0.005	0.038	0.002	0.011	0.060
Phishing	0.993	0.983	0.900	0.015	0.026	0.152	0.007	0.010	0.039
Jailbreak	0.920	0.705	0.940	0.142	0.543	0.104	0.067	0.262	0.047
Toxicity	0.807	0.530	0.662	0.370	0.602	0.432	0.185	0.297	0.177
RAG relevance	0.650	0.625	0.620	0.413	0.466	0.628	0.055	0.117	0.279
Model routing	1.000	0.639	0.977	0.000	0.494	0.040	0.000	0.089	0.014
Typed decisions	0.751	0.766	0.736	0.427	0.400	0.366	0.209	0.213	0.047
MASSIVE (51 languages, macro)	0.910	0.382	0.890	0.136	0.987	0.161	0.058	0.418	0.064
Macro, 11 suites	0.850	0.702	0.770	0.241	0.444	0.359	0.099	0.164	0.145

Kai runs/a6, weights a211cc70, cpu f32
harness a1a4a73, benchmarks efd4d91

Table 3: Accuracy, Brier score and ECE on the frozen harness, per suite, for Kai, Laya and Jev. MASSIVE is the macro over its 51 languages (Table 6); the macro row is over the other eleven suites. Bold: the best accuracy of the three in a row among backends that did not train on that suite’s test rows. †: Laya’s card lists the suite’s dataset in its mixture and the harness draws the suite from that dataset’s train split; the clean macro omits those suites. Written by **bench tex**; the last row names the revisions.

The gate. A checkpoint replaces a baseline in a deployment only when **bench gate** accepts it: on up to 100 drawn cases per harness and extension suite and their robustness transforms, no compared suite may lose more than 0.02 accuracy or gain more than 0.03 ECE, whatever the macro average, and for Laya the native runtime’s parity must hold. The first stage-A checkpoint was rejected: macro accuracy 0.639 over the 62 suites against Laya’s 0.438, parity held, and seven suites regressed, RAG relevance, typed decisions, Banking77 and jailbreak among them [measured] (hanzoai/benchmarks 9aa6a46). Stage a6 is rejected against Laya on one suite, support triage, and against Jev on jailbreak, typed decisions, toxicity (its ECE on the drawn cases, at 0.807 accuracy to Jev’s 0.662 over the suite) and eight of the 51 MASSIVE languages [measured]; so it does not replace either baseline in a deployment, and this paper claims no such thing.

8.2 Capabilities the baselines lack by construction

These suites are hanzoai/benchmarks decision/capability, in the harness’s case format; every backend gets the same cases. The baselines’ kept runs are at 4e57fb2.

Cardinality.

One choice among K options: 4 and 16 (the gold Banking77 label with its nearest labels by word overlap), 77 (Banking77), 150 (CLINC150), and 1,000 to 100,000 (WordNet: a definition to be matched to its concept among co-hyponyms and filler). An unanswered question counts as wrong. Laya rejects a question whose option markers fall past its context; Jev refuses more than 255 options. Laya’s authors’ recipe for many options is a top-20 shortlist from an encoder, which caps accuracy at the shortlist’s recall; the table reports every model on the full option set, where Kai’s retrieval keeps the mass it cuts (Section 3.2).

Joint coherence.

Two families of synthetic programs, 200 states each, ask five dependent questions of one state (a vehicle’s fault, severity, readiness, action and review; a source selection’s option, evidence, risk, readiness and review), with gold computed from a latent world by a policy the state spells out, so every gold answer keeps every rule. The baselines answer each question independently; Kai runs the refinement of Section 3.3 over a decoder stage a6 has not trained, and loses this suite to Jev (0.306 per variable to 0.837, and a rule broken in every state before projection): the decoder is what stage C trains. Per-variable accuracy, the share of states with all five right, and the share whose answers break a rule, as answered and after the same exact projection applied to every backend’s marginals.

Invariance.

200 Banking77 questions of 16 options asked in five orders: the share whose top answer changes and the largest probability change; 400 `noul` questions asked with the two sides’ descriptions exchanged, and again with the statement negated: the share whose answer mirrors. Proposition 2 predicts no flips and exact mirrors under the exchange for Kai, up to rounding; under negation a mirror is learned, not guaranteed, for every backend.

Media.

Human activity from smartphone inertial signals, a turbofan’s remaining life from C-MAPSS telemetry, and a valve’s condition from its sound (MIMII): the baselines read text renderings of the signals, Kai the records through its adapters when the checkpoint reads evidence, else the same rendering, which the table records.

Conformal sets.

Split-conformal prediction sets [39] at a 90% target over every answer on the frozen suites: coverage and mean set size.

Refresh.

A study program re-run after one evidence record changes: the questions put to a model, against a cold run.

Deployment.

Whether the backend answers inside a sandbox with the network denied: Laya does and Jev cannot, and every Jev call sends the state to a third party [measured].

8.3 Speed

A call is one state with N questions, timed from tokenizing to probabilities on the host, the first call at each N discarded, percentiles nearest-rank (**bench speed**). Kai and Laya run on the same Rust runtime, device and batch sizes, their calls interleaved, on Metal in F32 and on CPU, where the runtime serves Kai’s encoder projections in Q8_0 and Laya’s in F32; Jev is one Decisions API call over the network, split into the largest chunks it accepts, eight in flight, when it refuses the whole. The table’s last row names the Laya checkpoint timed: Laya’s multilingual checkpoint shares Kai’s encoder, mmBERT-base, so against it the difference is the head and the layout, while its English and typed-decisions checkpoints are ModernBERT-large, 28 layers of width 1,024 against 22 of width 768. Kai’s advantage, where it has one, comes from what Laya’s layout cannot share: each question’s sequence holds its question and the state but no options, each distinct option is encoded once for every question that lists it, and each question’s head costs the options it scores, not a

	Kai	Laya	Jev
<i>Cardinality</i>			
Accuracy, $K = 4$	0.685	0.777	0.905
Accuracy, $K = 16$	0.395	0.670	0.853
Accuracy, $K = 77$	0.255	0.425	0.843
Accuracy, $K = 150$	0.287	0.610	0.940
Accuracy, $K = 1,000$	0.005	0.000	0.000
Accuracy, $K = 10,000$	0.000	0.000	0.000
Accuracy, $K = 100,000$	0.000	0.000	0.000
Rejected, $K = 1,000$	0.000	1.000	1.000
Rejected, $K = 10,000$	0.000	1.000	1.000
Rejected, $K = 100,000$	0.000	1.000	1.000
<i>Joint coherence, five dependent questions</i>			
Accuracy	0.306	0.336	0.837
Exact	0.000	0.005	0.438
Violations	1.000	0.637	0.247
Accuracy, projected	0.341	0.328	0.879
Exact, projected	0.010	0.040	0.560
<i>Invariance</i>			
Permutation, flip	0.000	0.255	0.060
Permutation, max Δp	0.000	0.996	0.470
Noul sides swapped, mirror	0.915	0.205	0.062
Noul negated, mirror	0.430	0.300	0.882
<i>Media, accuracy</i>			
Human activity (HAR)	0.160	0.203	0.153
Turbofan life (C-MAPSS)	0.330	0.170	0.300
Valve sound (MIMII)	0.500	0.500	0.840
<i>Conformal sets, 90% target</i>			
Coverage	0.901	0.901	0.901
Mean set size	2.498	6.801	1.304
Refresh: questions asked / full	6 / 101	100 / 100	100 / 100
Runs offline	yes	yes	no
Kai runs/a6, weights a211cc70, cpu f32 harness a1a4a73, benchmarks efd4d91			

Table 4: Capabilities, Kai against Laya and Jev: cardinality, joint coherence, invariance, media, conformal sets, refresh and offline operation. Written by `bench tex`; the last row names the revisions.

N	Kai		Laya		Jev, network		Questions/s		
	p50	p95	p50	p95	p50	p95	Kai	Laya	Jev
<i>Metal</i>									
1	17.8	24.7	15.6	16.5	133	546	49.3	66.9	5.72
10	45.6	84.8	122	175	155	221	206	78.3	62.9
50	135	227	604	617	202	275	315	83.4	233
100	351	376	1,263	1,324	343	556	293	79.4	267
500	1,264	1,394	5,370	5,756	515	2,193	395	93.1	726
1,000	2,440	2,502	10,315	10,920	602	1,077	411	95.8	1,489
<i>CPU</i>									
1	139	147	214	219	133	546	8.32	4.99	5.72
10	365	399	1,622	2,025	155	221	28.5	5.83	62.9
50	920	980	7,492	7,818	202	275	53.5	6.58	233
100	1,720	1,767	15,391	16,170	343	556	57.6	6.40	267
500	43,152	44,820	178,544	182,739	515	2,193	11.7	2.80	726
1,000	80,165	80,165	329,783	329,783	602	1,077	12.5	3.03	1,489
Kai runs/a6, encoder 768 wide, 22 layers									
harness a1a4a73, benchmarks efd4d91									
timed F32 on Metal and on the CPU, Kai's CPU encoder projections Q8_0									
Laya laya-multilingual: hanzoai/kai-1-multilingual@3119843b, encoder 768 wide, 22 layers									
timed calls per N: Metal 10; CPU 3, 1 at N = 1,000									
dbc (Apple M4 Max, 128 GB)									

Table 5: Latency per call (ms, p50 and p95) and questions per second, by questions per state, on Metal and on CPU. Written by `bench tex`; the last row names the machine and revisions.

sequence holding all of them. The stage-A checkpoints use the per-question layout (Section 3.1), so these timings do not include encoding the state once for all N questions, which the shared layout adds. For reference, Laya's kept Python run on an M1 Max took 53 ms at $N = 1$ and 43.9 s at $N = 1,000$, and Jev 203 ms and 730 ms at \$0.0126 per 1,000-question call [\[measured\]](#) (4e57fb2).

9 Limits

What is not measured. Until `bench tex` writes the capability and speed tables for the final checkpoint, this paper claims no capability or speed figure for it. Stage B’s adapters and stage C’s decoder have code, data and stage files but no trained checkpoint, so the media rows measure a text rendering unless the scored checkpoint reads evidence, and the joint rows measure refinement over the factorized head, not the slot decoder. The Enso evaluation is a design.

What is not built. Latents kept across calls, refinement inside the program executor, the program form of the wire, the closed-form weight-change bound, sampled flip probabilities, rejecting a narrative that states an uncited number, and distilling refinement into one or two passes are [proposed].

What the harness cannot show. Kai trains on the train split of every harness source, so its leads over a model that did not are in-distribution; with Jev’s training data unknown, no table here measures generalisation to a new task. Laya’s typed-decisions score rests on templates shared with half the test set (Section 5.2), and the harness’s MASSIVE suites hold 100 utterances per language, so a language’s accuracy moves by 0.01 per question. The capability suites were built by us, for properties Kai was designed to have; they show what the baselines cannot do by construction, not that Kai does it well, which only their Kai rows say.

What stage A leaves open. The collapse of Section 5.1 is located (encoder, full mixture under a $7\times$ outer step) but not attributed to a part of the mixture; the fix removes the aggressive step and restarts from Laya’s weights rather than explaining it. Local SGD at a gentle outer step may still move the encoder further than one process does at the same learning rates, and only the harness and the context similarity say whether it did.

What stage a6 leaves open. Support triage fell under repetition while its validation rose, so the harness, not validation, must pick a mixture. The joint-coherence suite is lost outright until stage C trains the decoder. The serving modes of the design, a decision over a cached latent in microseconds and an incremental encoder over evidence chunks, are designed and partly built (the cache of question, state and option latents is exact and tested), not measured; every latency in this paper is a full encode per call.

Known failure modes of Laya. Its authors report that boolean-word labels can be followed instead of their descriptions, negated requests answered as unnegated, `noul` answers driven by the labels rather than the state, and a score’s first level rarely picked by the multilingual checkpoint [7] [reported]. A program’s tests should add a negated and a relabelled variant of each case, which Kai’s robustness transforms do in training.

Calibration is assumed, then checked. Thresholds, abstention, conformal sets and flip probabilities take probabilities at their word; a shift in the evidence distribution breaks fitted temperatures and pinned calibrations alike, and only shadow records detect it.

Adversarial state beyond the lattice. Proposition 5 protects constraints, not preferences: an adversary can push routing toward a cheaper model or completion toward an early finish. Those

need deterministic backstops, such as requiring a passing test run in the transcript. Deliberate friction is bounded by policy, not prevented.

The study model has assumptions. Additive scores need preferentially independent criteria; one-parameter flips hold the rest fixed; assumption sweeps are a grid; sampled flip probabilities treat cells as independent, which fails when two read the same evidence. A study’s figures are its sources’; the JLTV trade cites a company figure for the hybrid’s fuel economy, and its package says so rather than resolving it.

10 Related work

Typed decision models. TypeSafe’s Jev [37] introduced the typed interface adopted here, served through OpenRouter [30]; its design notes [20] argue for typed state and permissions as decisions. Laya [7] is an open encoder implementation trained with proper scoring rules [14].

Encoders that score options. Laya’s joint head is a cross-encoder, as in passage re-ranking [28] and entailment-based zero-shot classification [43], after BERT [10]. Encoding candidates independently, caching them, and letting them attend to a context is the poly-encoder [18] and, per token, late interaction [23]; Kai adds a law per type, a cumulative-link model for ordinal answers [26], and retrieval that keeps the mass it cuts.

Parallel refinement. Masked discrete diffusion [3], mask-predict [13] and MaskGIT [5] predict masked tokens in parallel and re-predict the uncertain ones. Kai’s unit is a typed variable with its own domain, ordered by the program’s DAG and projected onto its rules.

Multimodal front ends. Whisper [31] and SigLIP [44] give frozen audio and image features; time-series models [8, 2] give forecasts, which enter Kai as evidence, not answers.

Structured output and guards. Constrained decoding [41] removes parse failures, not the generation, and its probability is a sequence’s, not a distribution over the caller’s options. Tool-using agents [42, 33] make the decisions of Table 2 inside the generative loop. Guard models such as Llama Guard [19] judge text against a harm taxonomy; Section 6 places any guard below policy.

Calibration and abstention. Temperature scaling [15], the reject option [6], selective classification [12] and conformal prediction [39] are the theory behind fitted temperatures, abstention to ASK and the prediction sets of Table 4.

Training across machines. Local SGD with an outer Nesterov step is DiLoCo [11]; the outer step’s effective multiplier (Proposition 4) is the quantity its authors tune. Carrying the rounding of a compressed update into the next is error feedback [34, 21]; bf16 compute over F32 masters is mixed-precision training [27]. Kai’s trainer adds exact bookkeeping: join-order summation, a broadcast change that every copy applies bit for bit, and resume equal to the uninterrupted run.

Contamination. Near-duplicate detection by min-hashed shingles [4] is the standard tool for deduplicating training corpora against each other and against benchmarks [24]. Kai’s barrier adds sampled-span containment, which finds a test item embedded in a longer training text, and treats evaluation as a role that no license decision admits.

Authority and decision analysis. Enforcing the join of two verdicts is the lattice model of information flow [9] applied to decisions, and a policy evaluated before every action is a reference monitor [1]. Package scores are additive multi-attribute utility [22]; weight sensitivity is studied by Triantaphyllou and Sánchez [35], global sensitivity by Saltelli et al. [32], and what is worth resolving by the value of information [17].

11 Conclusion

Most small decisions do not need a generative model. A typed question over evidence, answered by an encoder with a law per type, gives a distribution a caller can threshold, record and replay. Encoding options alone removes the limits a shared sequence puts on option count, order and question count; decoding a program’s questions as typed variables, in waves bounded by depth and projected onto rules, makes their answers coherent; joining every model verdict with a deterministic policy makes safety the policy’s, with the cost of trusting the model counted exactly in shadow.

Training Kai taught as much as designing it. Local SGD across three kinds of accelerator works and is exact to the bit, but an outer step that multiplies a round’s delta seven times, over a mixture this broad, cost the encoder its ability to relate two texts while every single-text task improved; only measurements aimed at the encoder itself found it. A barrier against the test set is only as good as its notion of “the same”: half of one baseline’s test set had same-wording siblings in its training data, and measuring the halves apart showed the siblings did not help it. Both are written down here with the runs that show them.

Kai’s case does not rest on winning every benchmark: after stage a6 Laya leads on support triage and AG News, where its mixture holds the sources, Jev on jailbreak and by a point and a half on typed decisions through its typed checkpoint, and Kai on the other eight, MASSIVE among them; Kai enters this paper only through the tables its benchmark writes. It rests on what a hosted model cannot offer: a pinned revision, deployment where the data live, no per-call fee, a runtime linked into the system it governs, a package per run that says what was decided, from what, and what would change it, and, in Enso, less generation per successful task, which Section 7.4 is designed to measure.

References

- [1] James P. Anderson. Computer security technology planning study. Technical Report ESD-TR-73-51, U.S. Air Force Electronic Systems Division, 1972.
- [2] Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, et al. Chronos: Learning the language of time series. *Transactions on Machine Learning Research*, 2024.
- [3] Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. In *Advances in Neural Information Processing Systems*, 2021.
- [4] Andrei Z. Broder. On the resemblance and containment of documents. In *Compression and Complexity of Sequences*, pages 21–29, 1997.
- [5] Huiwen Chang, Han Zhang, Lu Jiang, Ce Liu, and William T. Freeman. MaskGIT: Masked generative image transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.
- [6] C. K. Chow. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1), 1970.

- [7] Convai Innovations. Laya: a multilingual, non-autoregressive decision engine. <https://github.com/NandhaKishorM/laya> and <https://huggingface.co/convaiinnovations/laya>, 2026. Version 0.3.20, Apache-2.0; model card accessed 2026-09-24.
- [8] Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. A decoder-only foundation model for time-series forecasting. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [9] Dorothy E. Denning. A lattice model of secure information flow. *Communications of the ACM*, 19(5), 1976.
- [10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, 2019.
- [11] Arthur Douillard, Qixuan Feng, Andrei A. Rusu, Rachita Chhaparia, Yani Donchev, Adhiguna Kuncoro, Marc’Aurelio Ranzato, Arthur Szlam, and Jiajun Shen. DiLoCo: Distributed low-communication training of language models. arXiv:2311.08105, 2023.
- [12] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, 2017.
- [13] Marjan Ghazvininejad, Omer Levy, Yinhan Liu, and Luke Zettlemoyer. Mask-predict: Parallel decoding of conditional masked language models. In *Proceedings of EMNLP-IJCNLP*, 2019.
- [14] Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477), 2007.
- [15] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, 2017.
- [16] Hanzo AI Research. A temporal assertion graph: Valid-time and transaction-time reads, schema as assertions, and erasure in an append-only store. <https://github.com/hanzoai/papers>, 2026. 2026-09-22.
- [17] Ronald A. Howard. Information value theory. *IEEE Transactions on Systems Science and Cybernetics*, 2(1), 1966.
- [18] Samuel Humeau, Kurt Shuster, Marie-Anne Lachaux, and Jason Weston. Poly-encoders: Architectures and pre-training strategies for fast and accurate multi-sentence scoring. In *International Conference on Learning Representations*, 2020.
- [19] Hakan Inan et al. Llama guard: LLM-based input-output safeguard for human-AI conversations. arXiv:2312.06674, 2023.
- [20] Jev engineering for coding agents: The TypeSafe founder’s blueprint for building with Jev, 2026. Working note, September 2026. A synthesis of design notes by Diogo Almeida (TypeSafe), independently compiled; not affiliated with or endorsed by TypeSafe.
- [21] Sai Praneeth Karimireddy, Quentin Rebjock, Sebastian U. Stich, and Martin Jaggi. Error feedback fixes SignSGD and other gradient compression schemes. In *International Conference on Machine Learning*, 2019.
- [22] Ralph L. Keeney and Howard Raiffa. *Decisions with Multiple Objectives: Preferences and Value Trade-offs*. Wiley, 1976.
- [23] Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020.
- [24] Katherine Lee, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. Deduplicating training data makes language models better. In *Annual Meeting of the Association for Computational Linguistics*, 2022.
- [25] Marc Marone, Orion Weller, et al. mmBERT: A modern multilingual encoder with annealed language learning. <https://huggingface.co/jhu-clsp/mmBERT-base>, 2025.

- [26] Peter McCullagh. Regression models for ordinal data. *Journal of the Royal Statistical Society: Series B*, 42(2):109–142, 1980.
- [27] Paulius Mikićevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations*, 2018.
- [28] Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. arXiv:1901.04085, 2019.
- [29] Object Management Group. OMG systems modeling language (SysML), version 2. <https://www.omg.org/sysml/>, 2025.
- [30] OpenRouter. Decisions API (alpha): POST /api/alpha/decisions. <https://openrouter.ai>, 2026. Accessed 2026-09-24.
- [31] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- [32] Andrea Saltelli, Marco Ratto, Terry Andres, Francesca Campolongo, Jessica Cariboni, Debora Gatelli, Michaela Saisana, and Stefano Tarantola. *Global Sensitivity Analysis: The Primer*. Wiley, 2008.
- [33] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.
- [34] Frank Seide, Hao Fu, Jasha Droppo, Gang Li, and Dong Yu. 1-bit stochastic gradient descent and its application to data-parallel distributed training of speech DNNs. In *Interspeech*, 2014.
- [35] Evangelos Triantaphyllou and Alfonso Sánchez. A sensitivity analysis approach for some deterministic multi-criteria decision-making methods. *Decision Sciences*, 28(1), 1997.
- [36] typed-decisions. <https://huggingface.co/datasets/LocalLLaMA/typed-decisions>, 2026. 400 cases, 2,000 decisions, four workflows; test split.
- [37] TypeSafe. Jev. Served as typesafe/jev-1.13 through OpenRouter, 2026. Typed decision model; listed on OpenRouter 2026-09-24.
- [38] U.S. Army. ARM26BX06-NV012: Agentic-AI, schema-driven decision management for auditable studies and acquisition decisions. <https://armysbir.army.mil/topics/agentic-ai-schema-driven-decision-management/>, 2026. SBIR topic; accessed 2026-09-24.
- [39] Vladimir Vovk, Alexander Gammerman, and Glenn Shafer. *Algorithmic Learning in a Random World*. Springer, 2005.
- [40] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, Nathan Cooper, Griffin Adams, Jeremy Howard, and Iacopo Poli. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. arXiv:2412.13663, 2024.
- [41] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. arXiv:2307.09702, 2023.
- [42] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
- [43] Wenpeng Yin, Jamaal Hay, and Dan Roth. Benchmarking zero-shot text classification: Datasets, evaluation and entailment approach. In *Proceedings of EMNLP-IJCNLP*, 2019.
- [44] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023.

A The wire

A request to `POST /v1/decisions`, in OpenRouter's Decisions schema:

```
{
  "model": "kai",
  "state": "Hi, we were billed twice for March. Please refund the duplicate today or we will cancel our plan.",
  "questions": {
    "department": {
      "type": "choice",
      "instructions": "Which department should handle this?",
      "criteria": {
        "billing": "invoices, payments, refunds",
        "technical": "bugs, outages, system errors",
        "other": "everything else"
      }
    },
    "urgency": {
      "type": "score",
      "instructions": "How urgent is this?",
      "criteria": ["not urgent", "soon", "blocking"]
    },
    "churn_risk": {
      "type": "noul",
      "instructions": "Does the user threaten to cancel or leave?"
    }
  }
}
```

The response carries one typed answer per question: `confidence` is κ (Equation (1)), `answer_confidence` is $p_{\max}$, and other Hanzo fields are omitted. The values are illustrative, not measured.

```
{
  "id": "dec_...",
  "model": "kai",
  "provider": "hanzo",
  "answers": {
    "department": {
      "type": "choice", "choice": "billing",
      "probabilities": {"billing": 0.90, "technical": 0.06, "other": 0.04},
      "confidence": 0.85, "answer_confidence": 0.90
    },
    "urgency": {
      "type": "score", "score": 1.75,
      "legend": {"0": "not urgent", "1": "soon", "2": "blocking"},
      "probabilities": {"0": 0.05, "1": 0.15, "2": 0.80},
      "confidence": 0.70, "answer_confidence": 0.80
    },
    "churn_risk": {
      "type": "noul", "noul": 0.85,
      "confidence": 0.70, "answer_confidence": 0.85
    }
  },
  "usage": {"input_tokens": 164, "output_tokens": 0, "cost": 0}
}
```


Changing "model" to "typesafe/jev-1.13" sends the same request to Jev through Open-Router; nothing else changes.

B A Decision Program

agent.command-risk@1, as in program/programs [\[in the code\]](#), pinned to stage a5's calibration:

```
{
  "id": "agent.command-risk@1",
  "description": "Before an agent runs a shell command: allow it, ask the user, or deny it.",
  "model": "kai",
  "mode": "advisory",
  "calibration": "cal_47711d44267fdbdd",
  "questions": {
    "verdict": {
      "type": "choice",
      "instructions": "Should the agent run this shell command?",
      "criteria": {
        "allow": "read-only, or confined to the working repository",
        "ask": "writes outside the repository, installs software or touches the network",
        "deny": "reads or sends secrets, keys, ~/.ssh or .env files, or destroys data"
      }
    },
    "secret_exposure": {"type": "noul", "instructions": "Could this command read, print or send credentials, keys or tokens?"},
    "production_impact": {"type": "noul", "instructions": "Could this command change or break a production system or shared data?"},
    "network": {"type": "noul", "instructions": "Does this command send or fetch data over the network?"}
  },
  "thresholds": {"verdict": 0.5, "secret_exposure": 0.3, "production_impact": 0.4, "network": 0.5},
  "verdict": {
    "from": "verdict",
    "escalate": {"secret_exposure": "deny", "production_impact": "ask", "network": "ask"}
  }
}
```

The verdict question abstains to ASK below $\kappa = 0.5$; each escalation raises the verdict when its noul reaches its threshold; the result joins the policy's verdict when the program runs enforced.

C MASSIVE by language

Language	Kai	Laya	Jev	Language	Kai	Laya	Jev
af	0.900	0.340	0.870	ko	0.920	0.470	0.940
am	0.850	0.150	0.850	lv	0.890	0.300	0.850
ar	0.930	0.460	0.870	ml	0.870	0.280	0.910
az	0.950	0.360	0.880	mn	0.880	0.160	0.840
bn	0.900	0.450	0.860	ms	0.920	0.270	0.920
cy	0.860	0.160	0.760	my	0.950	0.160	0.890
da	0.920	0.460	0.880	nb	0.920	0.500	0.860
de	0.920	0.530	0.939	nl	0.930	0.400	0.900
el	0.930	0.440	0.940	pl	0.930	0.470	0.920
en	0.940	0.820	0.920	pt	0.920	0.440	0.900
es	0.920	0.530	0.930	ro	0.950	0.370	0.940
fa	0.950	0.510	0.950	ru	0.980	0.570	0.940
fi	0.930	0.300	0.890	sl	0.890	0.330	0.890
fr	0.930	0.600	0.900	sq	0.920	0.290	0.760
he	0.910	0.370	0.940	sv	0.930	0.480	0.940
hi	0.920	0.460	0.950	sw	0.830	0.130	0.700
hu	0.870	0.340	0.920	ta	0.910	0.310	0.920
hy	0.870	0.250	0.840	te	0.860	0.220	0.910
id	0.930	0.360	0.930	th	0.930	0.480	0.940
is	0.870	0.330	0.840	tl	0.890	0.300	0.850
it	0.930	0.470	0.960	tr	0.900	0.410	0.920
ja	0.970	0.640	0.970	ur	0.920	0.420	0.900
jv	0.870	0.160	0.750	vi	0.930	0.330	0.890
ka	0.830	0.150	0.880	zh-CN	0.950	0.650	0.940
km	0.850	0.200	0.810	zh-TW	0.960	0.610	0.930
kn	0.900	0.300	0.870				

Kai runs/a6, weights a211cc70, cpu f32
harness a1a4a73, benchmarks efd4d91

Table 6: MASSIVE intent accuracy by language (ISO 639 codes), 100 utterances each, 20 options; random is 0.05. Written by **bench tex**; the last row names the revisions.